

CW2.1: Part 2 Take-home Assignment

Preamble

Mode: **Group** assessment. Students work in groups of four or five students; groups **remain the same** as for Phase 1.

Type: Formative assessment, prerequisite for In-Class Test 1 (CW1.2).

Deadline: **Wednesday 20th May (Week 12), 09:00 (China Standard Time).**

Feedback: General feedback will be provided to the entire cohort during the Week 8 lectures. Individual groups may request specific feedback via email and through in-person group consultation sessions (20 minutes per group, bookable through Learning Mall).

Description: You will solve the *Optimal Synchronised Schedule* problem described in the Overview document, using a graph-based approach, and compare your solution to the brute-force approach from Part 1. Detailed instructions are provided below.

Submission: Your group is required to submit the following **two (2)** documents to the dedicated “CW Phase 2 - Take-home Assignment” submission activity on Learning Mall:

- A **Java source code solution file**, named *Phase2_Group{X}_Soln.java*.
- A **group report** of no more than 6 pages (Appendix and cover page excluded), as a PDF file named *Phase2_Group{X}_Report.pdf*, and containing the following sections:
 - A detailed explanation of your graph construction, including what the vertices and edges represent, and how the graph is constructed from the input map;
 - An analysis of the cost of constructing the graph, measured in both time and space complexity using the Big-Oh notation from the lectures;
 - An explanation of your graph-based solution to the Synchronised Schedule problem, with details of the algorithm applied, and a discussion of the solution’s optimality;
 - An analysis of the time and space complexities of your solution, using Big-Oh notation from the lectures;
 - A comparison between the graph-based solution in Part 2 and the brute-force approach in Part 1.
 - Where applicable, an appendix detailing AI usage (see *AI Usage Statement*).

Group submissions settings have been enabled on Learning Mall, so only one submission per group is required. Use the groupwork submission cover sheet as cover page for your report.

AI Usage Statement: Generative artificial intelligence (GenAI) is a rapidly evolving technology with implications for graduate study. It is important to note that the **University's academic integrity policy has not changed**. For the take-home part of this project, you are permitted to make **substantive use of AI**. You may employ AI tools to assist you in **any part of solving the problem**, including but not limited to brainstorming solution ideas, generating Java code, and assistance with report-writing. However, the summative assessment for this coursework will be an on-site in-class test based on your take-home work, during which the use of AI tools will be strictly prohibited. As such, you must ensure you have a complete understanding of your solution, irrespective of any AI usage in developing it. In the appendix of your report, you should provide a full disclosure of your AI usage, including the tool name, version, and access date, and a detailed description of the AI's role, e.g.:

ChatGPT-5, accessed 10th May, 2026, generated Java code for *<insert functionality here>* based on a description of the task in natural language.

Late Submission: As this is a formative assignment, there will be no penalty for late submission. However, access to the summative in-class test will be limited to students whose group has a valid submission for this assignment on Learning Mall.

Academic Integrity Statement: This assignment is group work. Please refer to the Overview document's statement on *Collaboration and Academic Integrity*. Plagiarism (e.g., copying materials from other sources without proper acknowledgement) is a serious academic offence. Plagiarism, collusion, copying, unauthorised assistance, dishonest use of data, and academic offences involving AI will not be tolerated and will be dealt with in accordance with the University Academic Integrity Policy. Submitting work created by others, whether paid for or not, is a serious offence, and will be prosecuted vigorously. The use of AI tools beyond the scope outlined above would also be considered in breach of the University Academic Integrity Policy, and dealt with accordingly.

Detailed solution and report instructions

The remainder of this document is dedicated to providing you with some detailed guidance on how to solve your assigned task (i.e., the coding aspects), and what to include in the report. Note that this is intended for guidance only. Variations on the different constructions, algorithms, and functions are possible, within the constraint that your solution must employ a graph-based approach to solving the problem.

1. Preliminaries

As in Phase 1, a hospital map is encoded as an array of Strings, where each String represents a row of the map (all rows having equal length). Recall the following symbols used.

#	Wall / blocked cell
.	Free cell; there are P free cells in total
A	Starting position S_A of Robot A
B	Starting position S_B of Robot B
a	Target T_A of Robot A
b	Target T_B of Robot B

You may reuse your free cell encoding from Phase 1, where the set of free cells are indexed by 0, ..., P-1 using arrays. You may also reuse any move computation functions such as the `computeSuccessors(int identifier)` method that returns the list of identifiers of free cells that can be reached from the cell indexed by the given `identifier`.

2. Building the state graph

You should begin by converting the hospital map into a graph representation, such as a game configuration graph. You may make reasonable design choices: your graph may be directed or undirected, weighted or unweighted. However, your report must clearly explain what the vertices, edges, and edge weights mean.

Graph model example

A vertex represents one legal *state*, written as (a, b) , where a is the identifier of the free cell occupied by Robot A and b is the identifier of the free cell occupied by Robot B. In this representation, you only need to include states with $a \neq b$, since the robots are not allowed

to occupy the same cell after any tick. The starting state/vertex is (S_A, S_B) . The target state/vertex is (T_A, T_B) .

An edge represents a pair of legal moves from one state to another. That is, we put an edge from the state/vertex (a, b) to the state/vertex (a', b') in the graph if the pair of moves $(a \rightarrow a', b \rightarrow b')$ is legal. You may wish to label edges according to the actual moves made, in order to recover the entire schedule of moves. For example, an edge from (a, b) to $(\text{next}A, \text{next}B)$ may be labelled by the move pair (E, WAIT), meaning that Robot A moves East and Robot B stays in place during that tick.

For the original makespan problem, every useful edge has weight 1, because each edge represents exactly one tick. If you choose to use a weighted graph, explain what the weight represents. For example, a variant might charge different costs for moving, waiting, turning, or entering a high-traffic corridor.

Implementation hints

- Consider making the graph concise. For example, do not include the combined move (WAIT, WAIT) as an edge because this move is a self-loop from (a, b) back to (a, b) : it increases the makespan by one but changes nothing.
- You may wish to identify the states/vertices of the graph, building on the free-cell indexing from CW1.1. A simple state identifier is $\text{stateId} = a * P + b$. Values with $a == b$ can be marked invalid and skipped.
- A vertex record may store a, b , a list of outgoing edges, and optionally a visited flag, parent state, and parent move pair used during BFS.
- An adjacency list is usually the most natural representation. Avoid using a P^2 by P^2 adjacency matrix unless you have a specific reason, as it uses much more space.
- You may either build all edges before the search, or generate successors on demand during the search. If you generate successors on demand, explain that the graph is still the same conceptual state graph.

3. Complexity analysis - graph construction

You should analyse the cost of building the state graph from the previous section. Your analysis should include both time and space complexities, and use Big-Oh notation from the lectures. The key parameter is P , the number of free cells in the map. You may also use V and

E for the number of vertices and edges of the state graph, but your final analysis should then express these in terms of P. To aid you in your analysis, you may wish to consider the following questions:

- What is the cost in time and space of precomputing the single-robot successors for all free cells? How does the complexity change if successor lists are rebuilt every time they are needed instead of being precomputed?
- How many valid states can there be in terms of P?
- From one given state, how many pairs of legal moves are there (i.e., how many outgoing edges in the state graph)?
- Does your implementation generate successors once during graph construction, or repeatedly during graph traversals such as BFS? How do the different approaches impact the complexity of the solution?

4. Solving the Synchronised Schedule problem

Once the graph has been defined, the problem becomes a path-search problem. In the graph representation, a synchronised schedule is just a path from the starting state/vertex (S_A, S_B) to the target state/vertex (T_A, T_B) . You should consider using variants of Breadth-First Search (BFS), Depth-First Search (DFS), Dijkstra's algorithm or other suitable graph algorithms for this task. Note that in order to find an *optimal* synchronised schedule, your algorithm must find a *shortest path* from (S_A, S_B) to (T_A, T_B) .

Your code should therefore include a `solve(map)` function which prints a valid synchronised schedule for the given input map, and return with its corresponding makespan. If no synchronised schedule exists, the method should return -1. A schedule may be printed one tick per line, for example:

```
Tick 1: A=E, B=WAIT
```

Any equivalent unambiguous notation is acceptable.

Your report should explain which path-finding algorithm you applied, justifying your choice. It should then explain how the synchronised schedule and corresponding makespan are computed through this algorithm. Finally, it should include a discussion on your solution's *optimality*: does your algorithm compute just any valid synchronised schedule, or does it find the/an optimal one? If you claim that your solution produce the shortest schedule, you have to justify why this is indeed the case.

5. Complexity analysis - solution algorithm

You should analyse the cost of your graph-search solution to the Synchronised Schedule problem. Your analysis should include both time and space complexities, using the BigO notation from the lectures. Note that the analysis in this section should *only* consider the graph-search algorithm(s), independently of the cost of building the state graph (which was already analysed in Section 3). You may wish to first analyse the complexities in terms of V and E , then translate your bounds into expressions of P . To aid you in your analysis, you may wish to consider the following questions:

- What is the time complexity, in terms of V and/or E , of your chosen path-finding algorithm (BFS, DFS, Dijkstra, or other) on a graph stored with adjacency lists?
- What auxiliary data structures does the algorithm need, such as a queue, visited array, distance array, or parent array? What is the auxiliary space complexity of the algorithm in terms V and/or E ?
- How do the time and space complexities change after substituting V and E in terms of P ?
- What assumption, if any, are you making about the cost of obtaining or generating successors during the algorithm?
- What extra information needs to be stored if you want to reconstruct the actual synchronised schedule? How does this affect the space complexity?
- What is the cost of reconstructing the final path or schedule after the target configuration is found?

6. Comparison with Part 1 brute-force approach

Your discussion should explicitly compare the graph-based solution above with your brute-force solution from CW1.1. The two approaches may look different in code, but they are closely related. For example:

- CW1.1 computes `reachable[t][a][b]` for increasing t up to a fixed maximum search time T . This is essentially a level-by-level search in the corresponding state graph.
- The graph-based solution replaces the fixed three-dimensional time array with a standard graph traversal. BFS layers correspond to values of t : layer 0 is (S_A, S_B) , layer 1 is all states reachable at time $t=1$, and so on.

You should state which approach is most efficient from a time and/or space complexity perspective, and explain why (e.g., does the graph-based approach avoid duplicate work, and

if so, how?). Your comparison should discuss code structure as well as complexity: indexing cells, testing legal move pairs, storing predecessors, avoiding duplicate work, and printing an optimal schedule. You may also wish to return to your discussion of limitations of the brute-force approach in Part 1, and consider whether any of these have been solved through switching to a graph-based approach in Part 2. For example, why is the user-supplied time bound T no longer needed in Part 2?

Optional task: research and design challenge

This task is optional. It is intended for students who want to connect the coursework problem with current research and to think beyond the standard graph-search solution.

Research challenge

The two-robot hospital problem is a small instance of a wider research area usually called Multi-Agent Path Finding (MAPF), cooperative path-finding, or multi-robot path planning. Search the literature for state-of-the-art solutions to this problem, to variants of this problem, or to more general versions such as any number of robots, arbitrary graphs, weighted edges, directed edges, continuous time, or higher-dimensional grids.

- Find at least one paper or high-quality source that defines the general MAPF problem and its common objective functions, such as makespan or sum of costs.
- Find at least one algorithm that is not simply BFS over the full joint configuration graph. Possible keywords include Conflict-Based Search (CBS), A*, IDA*, ICTS, SAT-based MAPF, integer programming, prioritised planning, reservation tables, Push and Swap, and Push and Rotate.
- Explain which assumptions in the paper match this coursework and which assumptions are different. For example, do agents wait? Are swaps forbidden? Is the graph directed? Is the objective makespan or sum of costs?
- Discuss whether the method is complete, optimal, both, or neither. A complete method is guaranteed to find a solution when one exists. An optimal method is guaranteed to return a best solution according to the chosen objective.

Design challenge

Try to propose another solution to the coursework problem. It does not have to be graph-based. For example, you could encode the schedule as a SAT problem, an integer program, a constraint satisfaction problem, or a search with heuristics. You could also design a rule-

based method that tries to move robots along individual shortest paths while reserving cells in time. Your proposal should include a clear statement of its input, output, data structures, and limitations.

Submission instructions

If you choose to attempt this optional challenge, submit your research discussion and design proposal, consolidated into a single PDF file, as an additional document to the dedicated “CW Phase 2 - Take-home Assignment” submission activity on Learning Mall. Name your file *Phase2_Group{X}_OptionalTask.pdf*. Do not append this to the required report: submit it as a separate document.

For students interested in publication. If you believe your extension is genuinely new or experimentally interesting, talk to the course instructors. They can help you decide whether the idea is original, how to evaluate it properly, and whether it is suitable for a student research project or publication. Do not submit or publish coursework-derived materials without discussing academic integrity and authorship with the teaching team first.

Suggested starting points for optional reading

- [1] Stern et al. (2019), *Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks*. Available at: <https://arxiv.org/abs/1906.08291>
- [2] Sharon et al. (2015), *Conflict-Based Search for Optimal Multi-Agent Pathfinding*. Available at: <https://doi.org/10.1016/j.artint.2014.11.006>
- [3] Sharon et al. (2012), *Conflict-Based Search For Optimal Multi-Agent Path Finding* (AAAI version). Available at: <https://ojs.aaai.org/index.php/AAAI/article/view/8140>
- [4] Luna and Bekris (2011), *Efficient and Complete Centralized Multi-Robot Path Planning*. Available at: https://ai.dmi.unibas.ch/research/reading_group/luna-bekris-iros2011.pdf